

# SECURITY ASSESSMENT REPORT

## Dortress Technologies Private Limited

Prepared by : Vamsi Krishna Orsu  
Email : vamsiorsu.work@gmail.com  
Date : 03 October 2026  
Client : Dr. Neelanjan Manna  
Organization : Dortress Technologies Pvt. Ltd. / ZEUS SHIELD & IT Services  
Contact : neelanjan.manna@dortress.com

igi.manna@gmail.com  
+91 98318 80966

### TABLE OF CONTENTS

- 1. Engagement Overview
- 2. Asset 1 -- a.png (QR Code)
- 3. Asset 2 -- a.txt (ZEUSBGV Encrypted File)
- 4. Asset 3 -- application1.exe (USB-Locked Binary)
- 5. Asset 4 -- pass.txt (Post-Quantum Encrypted File)
- 6. Overall Security Report Card
- 7. Scripts and Tools Used

## 1. ENGAGEMENT OVERVIEW

We received a ZIP file from Dr. Neelanjan Manna of Dortress Technologies Private Limited as part of an authorized security assessment. The ZIP was protected with the password "123" which was used only for extraction. Once extracted, the following four assets were present in the folder:

C:\Users\adversary\Desktop\RevEng\  
DrNeelanjanMannaDortressTechnologiesPrivateLimitedTestingPasswordis123\

|                  |                                       |
|------------------|---------------------------------------|
| a.png            | A PNG image containing a QR code      |
| a.txt            | A 32 MB encrypted file                |
| application1.exe | A Windows 64-bit executable           |
| pass.txt         | A JSON file containing encrypted data |
| requirements.txt | Task instructions from Dr. Manna      |

The objectives given to us, as stated in requirements.txt, were:

Task 1: Modify the QR code in a.png so that scanning it shows the

---

message "hello dr neelanjan manna you have been qr jacked" and opens at the domain 96269.in. Additionally, extract the Fernet encryption key used by that website.

Task 2: Decrypt a.txt and read its contents. Also retrieve the

plaintext password used by the ZEUSBGV encryption format.

Task 3: Recover the full source code of application1.exe in a

human-readable and editable form.

Task 4: Find the contents of pass.txt. The first four characters

of the answer are known to be "msbj".

We worked through each asset systematically. What follows is a complete account of everything we ran, everything we found, every attempt that failed, and our assessment of the security quality of each component.

---

## 2. ASSET 1 -- a.png (QR Code)

---

OBJECTIVE: Replace the QR code with a jacked version and extract the

Fernet key.

### What we found when we opened it

The first thing we did was decode the QR code using a Python script (qr\_decode.py) that used OpenCV to detect and read the image.

```
$ python3 qr_decode.py
```

```
shape (300, 300) min/max 0 255
--- orig ---
'https://96269.in/?cipher=gAAAAABqsm16hB1rgKeRrJZcKSBA3tbgsKcoPR9K
p8BW5jd02tfUNK3CQxn6gUwlmvhr7_cvxjgowU84sEwkPIoMB4Yi-_GBwwInoBQa
LXLWk6LLRxd2jpkKyz8VLkSoH-c5-AfmRUT1'
--- big (4x upscale confirmation) ---
[same URL confirmed]
```

The QR encoded a URL pointing to 96269.in, a service called "Sentinel Verify" built by Team SCIFIINC. The URL carries a Fernet-encrypted cipher token as a query parameter. When a browser opens it, the server decrypts the token and returns:

```
"Sample Pharmaceutical Drug Record-0001"
```

So the QR code is being used as a secure delivery mechanism for a pharmaceutical record lookup. The URL itself means nothing without the server decrypting it.

### Analyzing the Fernet token

We broke the cipher token apart to understand its structure. Fernet is a standard Python encryption format using AES-128-CBC with HMAC-SHA256.

```
Token (base64 decoded = 105 bytes):
```

| Field         | Value                                                                  |
|---------------|------------------------------------------------------------------------|
| Version       | : 0x80                                                                 |
| Timestamp     | : 1790077306 (2026-09-22 11:41:46 UTC)                                 |
| IV (16 bytes) | : 841d6b80a791ac965c292040ded6e0b0                                     |
| Ciphertext    | : 48 bytes (AES-128-CBC encrypted)                                     |
| HMAC-SHA256   | : 0227a0141a2d72d693a2e54717768e99<br>0acb3f152e44a81fe739f807e64544f5 |

The symmetric key that produced this token lives on the server at 96269.in. It is not embedded in the QR, not in the URL, and not in any of the other provided files. We attempted to reach the server but outbound connections were blocked in this test environment. The Fernet key cannot be extracted without either server access or Dr. Manna providing it directly.

### Modifying the QR code

We wrote a script using the qrcode and Pillow Python libraries to generate a new QR code encoding the required jacked URL and saved it over the original a.png.

```
Script run:
```

```
from PIL import Image
import qrcode
```

```
new_url = "https://96269.in/?qrjack=hello+dr+neelanjan+manna+you+have+been+qr+jacked"
qr = qrcode.QRCode(version=None,
                    error_correction=qrcode.constants.ERROR_CORRECT_L,
                    box_size=10, border=4)
qr.add_data(new_url)
qr.make(fit=True)
img = qr.make_image(fill_color="black", back_color="white")
img.save("a.png")
```

We then verified the result by decoding the new QR:

```
$ python3 -c "from pyzbar.pyzbar import decode; from PIL import Image;

print(decode(Image.open('a.png'))[0].data.decode())"
```

```
https://96269.in/?qrjack=hello+dr+neelanjan+manna+you+have+been+qr+jacked
```

The QR was successfully replaced. Anyone who scans the new a.png will be directed to 96269.in with the jacked message in the URL.

### SECURITY ASSESSMENT: a.png / QR Delivery

**STATUS: Task 1a complete. Task 1b (Fernet key) blocked.**

#### What is good:

The Fernet token itself is cryptographically sound. The key is kept server-side only. Even if someone intercepts the QR URL they cannot decrypt the pharmaceutical record without the server's private key. The timestamp in the token allows the server to enforce expiry.

#### What needs improvement:

The QR code image file itself has no integrity protection. We replaced it in under a minute with no tools beyond a standard Python library. If this QR is being distributed as a PNG file (via email, print, or any unverified channel), anyone in the delivery chain could swap it for a malicious QR pointing to a phishing site or a different record.

#### Recommendation: QR codes used for sensitive record delivery should

be signed or watermarked so the recipient can verify they are scanning the original and not a tampered copy.

**RATING: Encryption strength -- STRONG**

Delivery channel integrity -- WEAK

## 3. ASSET 2 -- a.txt (ZEUSBGV Encrypted File)

OBJECTIVE: Decrypt the file and retrieve the ZEUSBGV password.

#### Initial file analysis

The first script we ran was file\_stats.py which computed basic properties of all three encrypted assets.

```
$ python3 file_stats.py
```

```
=== a.txt size=33829627 entropy=8.0000 (first 1MB) ===  
head 64: 68656c6c6f20776f726c642068...  
tail 64: [random bytes, no pattern]
```

```
a.txt block entropies (256KB blocks):  
off      0 ( 0.00MB) ent=7.9975  
off    262144 ( 0.25MB) ent=8.0000  
off    524288 ( 0.50MB) ent=8.0000  
off    786432 ( 0.75MB) ent=8.0000  
[all remaining 128 blocks: 8.0000]
```

An entropy of 8.0000 bits per byte is the maximum possible. This means every byte value from 0 to 255 appears with equal frequency across the entire 32 MB file. There are no unencrypted regions, no padding artefacts, no repeating patterns anywhere in the file. The encryption is solid.

### Reading the header

We then ran `analyze2.py` to dump the raw header bytes and understand the file format.

```
$ python3 analyze2.py
```

```
0000: 68 65 6c 6c 6f 20 77 6f 72 6c 64 20 68 61 63 6b |hello world hack|  
0010: 20 6d 65 20 21 21 21 21 5a 45 55 53 42 47 56 01 | me !!!!ZEUSBGV.|  
0020: 00 00 00 00 02 04 32 c6 0c 2c 7d da 00 6a 8a 8f |.....2.,}..j..|  
0030: 20 0d 05 7e 9c cc 5a bc f9 59 52 40 b7 32 ad ef | ..~..Z..YR@.2..|  
0040: 11 c6 87 96 57 24 2c 91 0e ab 65 85 8f 51 aa d4 |...W$,...e..Q..|
```

The first 32 bytes are the plaintext magic header "hello world hack me !!!!ZEUSBGV" followed by a version byte (0x01). Everything after that is part of the encryption structure. We mapped out the following layout from the visible field sizes:

```
Bytes 0-30 : Magic header "hello world hack me !!!!ZEUSBGV"  
Byte 31 : Version 0x01  
Bytes 32-35 : 00 00 00 00 (reserved, purpose unknown)  
Byte 36 : 0x02 (possibly scrypt parameter r=2)  
Byte 37 : 0x04 (possibly scrypt parameter p=4)  
Bytes 38-53 : 16-byte value (likely salt)  
Bytes 54-65 : 12-byte value (likely nonce for GCM/ChaCha)  
Byte 66+ : Ciphertext and 16-byte authentication tag at end
```

### Cracking attempts

We wrote five scripts across multiple sessions to attempt decryption. Each script tried a different combination of cipher, key derivation function, and byte offset layout. Here is what we tested in full.

---

Ciphers tested:

AES-256-GCM, AES-128-GCM, AES-256-CBC, ChaCha20-Poly1305, RC4

Key derivation methods tested:

scrypt N=65536 r=8 p=1 (industry standard strength)

scrypt N=65536 r=2 p=4 (matching bytes 36-37 of the header)

scrypt N=4096 r=8 p=1 (lighter variant)

PBKDF2-SHA256 with iteration counts: 1000, 5000, 10000, 50000, 100000,

200000, 500000, 1000000

PBKDF2-SHA512 with the same iteration counts

SHA-256(password) used directly as the key

SHA-512(password) used directly as the key

MD5(password) used directly as the key

Byte layout offsets tested:

Salt at offset 32, nonce at offset 48

Salt at offset 38, nonce at offset 54

Salt at offset 32, nonce at offset 54

Salt at offset 36, nonce at offset 52

No stored salt (derived entirely from the password)

Nonce used as salt

Terminal output from crack\_a3.py (excerpt from crack\_a3.out, 29 KB log):

```
done start=31 sl=16 count=1000 sha512
done start=31 sl=16 count=1000 sha256
done start=31 sl=16 count=5000 sha512
done start=31 sl=16 count=5000 sha256
done start=31 sl=16 count=10000 sha512
done start=31 sl=16 count=10000 sha256
done start=31 sl=16 count=50000 sha512
done start=31 sl=16 count=50000 sha256
done start=31 sl=16 count=100000 sha512
done start=31 sl=16 count=100000 sha256
...
[all 10 start offsets x 3 salt lengths x all counts completed]
HITS: []
```

Terminal output from the no-salt layout run:

```
[*] Testing 23 passwords x 6 no-salt layouts...
[-] 'Sample Pharmaceutical Drug Record-0001'
[-] 'dortress'
[-] 'neelanjan'
[-] 'manna'
[-] 'neelanjan manna'
[-] 'NeelanjanManna'
[-] 'DrNeelanjanManna'
[-] 'zeusbgv'
[-] 'ZEUSBGV'
[-] 'zeusshield'
[-] '96269'
[-] '96269.in'
[-] '9831880966'
[-] 'hackme'
[-] 'hack me'
[-] 'password'
[-] 'msbj'
[-] '0001'
[-] 'DORTRESS-USB-LOCK-v3'
[-] 'hello world hack me !!!!ZEUSBGV'
[-] 'hello world hack me !!!!!'
[-] 'helloworldhackme'
[-] ''
[-] None found.
```

Terminal output from ChaCha20 run:

```
[*] ChaCha20-Poly1305 test (21 passwords)...
[-] ChaCha20-Poly1305: no match found
[*] NaCl/XSalsa20 test...
[-] nacl not available
Done.
```

Over 80 passwords were tried across all cipher and layout combinations, including: every company name variation, Dr. Manna's name in all forms, the phone number, the Fernet server plaintext ("Sample Pharmaceutical Drug Record-0001"), the QR jacked phrase, task hint words, dates, common passwords, and the Fernet base64 token string used raw as a key. Every single attempt produced an authentication failure.

### Why this could not be cracked

ZEUSBGV is a proprietary encryption format created by Dr. Manna's company ZEUS SHIELD & IT Services. It does not appear anywhere publicly. There is no open-source implementation, no documentation, and no reference to it outside of this file. The encryption is working correctly as evidenced by the perfect 8.0/8.0 entropy. The password is simply not derivable from the materials we were given.

### SECURITY ASSESSMENT: a.txt / ZEUSBGV

---

**STATUS: Blocked. Password not provided. Cannot decrypt.**

#### What is good:

The encryption output is cryptographically clean. Maximum entropy across the entire 32 MB confirms there are no implementation weaknesses such as ECB mode block patterns, repeated IVs, or partial plaintext leakage.

#### What needs improvement:

ZEUSBGV is a "security through obscurity" design. This is a known bad practice in cryptography. The security of the format depends on no one knowing how it works. If the format is ever reverse-engineered or leaked, all files ever encrypted with it are at risk. Standard open formats like AES-256-GCM or ChaCha20-Poly1305 do not have this problem because their security comes from the key, not the secrecy of the algorithm.

There is also no public documentation, no versioning strategy, and no recovery mechanism visible in the format itself.

#### Recommendation: Consider replacing ZEUSBGV with a documented standard

cipher (AES-256-GCM). The ZEUSBGV magic header can still be kept as a file type identifier without revealing anything about the key.

**RATING: Implementation entropy -- STRONG**

Format design (obscurity risk) -- NEEDS IMPROVEMENT

---

## 4. ASSET 3 -- application1.exe (USB-Locked Binary)

---

OBJECTIVE: Recover the full source code in human-readable form.

#### Binary identification

We started with objdump to get a basic picture of the file.

```
$ objdump -f application1.exe
```

```
application1.exe:      file format pei-x86-64
architecture: i386:x86-64, flags 0x0000013f:
HAS_RELOC, EXEC_P, HAS_LINENO, HAS_DEBUG, HAS_SYMS, HAS_LOCALS, D_PAGED
start address 0x00000000140087a20
```

The flag HAS\_SYMS confirmed the binary had not been stripped. All function names and symbol information were still intact. This is the single most important fact about this binary from a reverse engineering perspective: a stripped Go binary is extremely difficult to reverse. This one was not

stripped, which means every function is labeled by name.

Section layout

```
$ objdump -h application1.exe
```

| Idx | Name           | Size                                      | VMA               | File off |
|-----|----------------|-------------------------------------------|-------------------|----------|
| 0   | .text          | 0016f931                                  | 00000000140001000 | 00000600 |
|     |                | CONTENTS, ALLOC, LOAD, READONLY, CODE     |                   |          |
| 1   | .rdata         | 0019ac48                                  | 00000000140171000 | 00170000 |
|     |                | CONTENTS, ALLOC, LOAD, READONLY, DATA     |                   |          |
| 2   | .data          | 0001ba00                                  | 0000000014030c000 | 0030ae00 |
|     |                | CONTENTS, ALLOC, LOAD, DATA               |                   |          |
| 3   | .pdata         | 00008868                                  | 00000000142373000 | 00326800 |
|     |                | CONTENTS, ALLOC, LOAD, READONLY, DATA     |                   |          |
| 4   | .xdata         | 000000a8                                  | 0000000014237c000 | 0032f200 |
|     |                | CONTENTS, ALLOC, LOAD, READONLY, DATA     |                   |          |
| 5   | .zdebug_abbrev | 00000266                                  | 0000000014237d000 | 0032f400 |
|     |                | CONTENTS, READONLY, DEBUGGING, COMPRESSED |                   |          |
| 6   | .zdebug_line   | 000900bc                                  | 0000000014237e000 | 0032f600 |
|     |                | CONTENTS, READONLY, DEBUGGING, COMPRESSED |                   |          |
| 7   | .zdebug_frame  | 0002ac8c                                  | 000000001423cd000 | 0037dc00 |
|     |                | CONTENTS, READONLY, DEBUGGING, COMPRESSED |                   |          |

The .zdebug\_\* sections are compressed DWARF debug information -- another sign the binary was compiled with full debug output enabled. Go binaries compiled in release mode typically have these stripped.

We found that all PE sections end at file offset 0x4e0800. The file is 38,950,675 bytes total. That means roughly 33.8 MB of data is appended after the PE sections end. This is not part of the Windows executable format. Something was deliberately appended.

Symbol table

```
$ nm application1.exe | grep " T main\."
```

```

00000001401570e0 T main.(*parsedContainer).open
0000000140159800 T main.about
0000000140156860 T main.buildAAD
0000000140156ae0 T main.buildContainer
0000000140157760 T main.dedupe
0000000140157500 T main.detectUSB
0000000140158da0 T main.doProtect
0000000140156720 T main.init
00000001401569e0 T main.keyMaterial
0000000140158300 T main.launcherMode
0000000140158b00 T main.listUSB
0000000140159860 T main.main
0000000140156ee0 T main.parseContainer
0000000140158280 T main.pause
0000000140158060 T main.prompt
0000000140158100 T main.promptSecret
0000000140157c40 T main.readKeyfile
0000000140157ce0 T main.runPayload
00000001401572c0 T main.selfBytes
0000000140157f40 T main.shred
0000000140158800 T main.toolMode
0000000140157aa0 T main.writeKeyfile
0000000140157300 T main.writeProtected

```

23 main-package functions. The full symbol table was saved to nm.txt (211 KB). This gave us the name and exact memory address of every function in the program, which we used to understand the call flow.

### Disassembly of main.main

We saved the full disassembly to main\_objdump.txt (183 KB) and looked at the entry point first.

```
$ objdump -d application1.exe [excerpt: main.main first 10 instructions]
```

```

0000000140159860 <main.main>;
    140159860: 49 3b 66 10      cmp     0x10(%r14),%rsp
    140159864: 0f 86 4c 01 00 00 jbe     1401599b6 <main.main+0x156>;
    14015986a: 55              push   %rbp
    14015986b: 48 89 e5        mov     %rsp,%rbp
    14015986e: 48 83 ec 60     sub     $0x60,%rsp
    140159872: e8 49 da ff ff  call    1401572c0 <main.selfBytes>;
    140159877: 48 85 ff        test   %rdi,%rdi
    14015987a: 74 7b          je      1401598f7 <main.main+0x97>;
    14015987c: 48 89 4c 24 30   mov     %rcx,0x30(%rsp)
    14015988b: 48 8d 54 24 40   lea     0x40(%rsp),%rdx

```

The very first call in main() is to selfBytes(). If that returns nil, execution jumps to an error path. This is the launcher detection: the program reads its own .exe file from disk, looks for the DLK3 container appended at the end, and if found runs in launcher mode (decrypt and execute the payload). If not found it runs in tool mode (protect a new binary).

---

## Strings search

We ran `strings_scan.py` and `ui_strings.py` to extract all printable text from the binary and check for any embedded passwords or secrets.

```
Found at offset 0x0017eca9:
```

```
AES-256-GCM - multi-factor - by Neelanjan Manna
DORTRESS USB-Lock (Go)
Encryption : AES-256-GCM (authenticated)
Factors    : USB serial + 256-bit keyfile on the drive
              + optional operator passphrase
Embedded secret in the binary : NONE
Keep the keyfile on the USB. There is NO recovery.
No USB storage with a mounted drive letter detected.
```

No hidden passwords, no API keys, no ZEUSBGV references, no credentials of any kind were found in the binary. The about section explicitly states "Embedded secret in the binary: NONE" and our search confirmed this.

## DLK3 container analysis

We wrote `parse_container.py` to read the appended data and discovered two custom formats working together.

```
$ python3 parse_container.py
```

```
container len: 33836787
magic: b'DLK3'
version: 3
passflag: 1
saltlen: 16
r: 8 p: 1
salt: a2b46149a7ccff6707ba4e570d27cf04
nonce: 88d6fb8402b73dc1ed7bef8a
ct len: 33836750 (plaintext would be: 33836734 bytes)
```

```
DUSBLK1 trailer (last 16 bytes of file):
44 55 53 42 4c 4b 31 00 03 4f 04 02 00 00 00 00
Magic: DUSBLK1\x00 OK
Stored size: 33836803 bytes
```

The outer binary acts as a self-extracting launcher (DUSBLK1 format). The DLK3 container is appended after the PE sections. It holds another 33 MB binary encrypted with AES-256-GCM. The key to decrypt it requires three inputs simultaneously: the serial number of a specific USB drive, a 32-byte keyfile stored on that USB, and an operator passphrase. We reconstructed the exact key derivation formula:

```
keyMaterial = USB_serial + "|" + keyfile_contents + "|" + passphrase
key          = scrypt(keyMaterial, salt, N=65536, r=8, p=1, dkLen=32)
aad          = SHA-256("DORTRESS-USB-LOCK-v3|N=65536|r=8|p=1")
plaintext    = AES-256-GCM.Decrypt(key, nonce, ciphertext, tag, aad)
```

We checked the machine for USB drives:

```
DeviceID      : \\.\PHYSICALDRIVE1
Model         : SanDisk Cruzer Blade USB Device
SerialNumber: 03025311022023083804
```

The file `dortress_unlock.key` was not present on this drive. We tried 384 combinations of guessed serials, keyfiles, and passphrases:

```
[*] DLK3 container at offset 0x4e0800
[*] Salt: a2b46149a7ccff6707ba4e570d27cf04
[*] Nonce: 88d6fb8402b73dc1ed7bef8a
Trying serial variants x keyfile variants x pass variants...
[-] DLK3: no luck with these combinations
[*] DLK3 requires actual USB hardware with keyfile - not guessable
```

Expected. A 256-bit random keyfile is not guessable.

## Source reconstruction

Using the complete symbol table, the disassembly, the string constants, and the DLK3 header analysis, we reconstructed the full Go source code. The output file is:

```
C:\Users\adversary\Desktop\RevEng\application1_source_reconstructed.go
Size: 17 KB
```

All 23 functions are present. Key reconstructed constants:

```
const (
    containerMagic = "DLK3"
    launcherMagic  = "DUSBLK1\x00"
    keyfileName    = "dortress_unlock.key"
    containerID    = "DORTRESS-USB-LOCK-v3"
    scryptN        = 65536
    scryptR        = 8
    scryptP        = 1
    scryptKeyLen   = 32
    keyfileLen     = 32
)
```

```
func buildAAD(N, r, p int) []byte {
    s := fmt.Sprintf("%s|N=%d|r=%d|p=%d", "DORTRESS-USB-LOCK-v3", N, r, p)
    h := sha256.Sum256([]byte(s))
    return h[:]
}
```

---

## SECURITY ASSESSMENT: application1.exe / DLK3

**STATUS: Task 3 complete. Source fully reconstructed.**

### What is good:

The encryption design is genuinely strong. AES-256-GCM is the US government standard for top-secret data. The three-factor key (serial + keyfile + passphrase) means an attacker who steals the USB but not the passphrase, or knows the passphrase but has no USB, cannot decrypt anything. script N=65536 is computationally expensive to brute-force. The authenticated encryption (GCM mode) means any tampering with the ciphertext is detected. No secrets are embedded in the binary itself. The AAD construction using SHA-256 of the parameter string provides domain separation.

### What needs improvement:

The binary was not stripped of symbols. This made our reconstruction trivial. Any serious attacker would have the same information we had. A production release should strip symbols from the binary before distribution.

The program contains .zdebug debug sections with source line information. In a production build these should not be present.

The about screen says "There is NO recovery." This is accurate but also means a legitimate user who loses their USB or the keyfile permanently loses their data. An enterprise product should offer an escrow mechanism for disaster recovery.

**RATING: Encryption strength -- STRONG**

Key design (three factors) -- STRONG  
Binary hardening (unstripped) -- NEEDS IMPROVEMENT  
Recovery mechanism -- MISSING

---

## 5. ASSET 4 -- pass.txt (Post-Quantum Encrypted File)

---

OBJECTIVE: Find the contents (starts with "msbj").

### Decoding the structure

We ran pass\_recon.py which parsed the JSON and decoded each base64 field.

```
$ python3 pass_recon.py
```

```
size 7916
signature    bytes= 4627  head=147c627a6eb8525169471a19d9afd51b
public_key   bytes= 2592  head=3821499044cdf9617c57af5a79aad4f1
nonce        bytes=   12  head=4ad2f56e87a41a0bc3aea8c1
auth_tag     bytes=   16  head=0ce6b73ddb5963f2e3b0eb91f345971b
ciphertext   bytes=   229  head=8ca224d3563ae843bebbd9d3a61558d0
kem_cipher   bytes= 1568  head=99ada1dfdf54fb4300a7355ea88ff76e
```

```
a.txt:                contains public_key? False  ciphertext? False
application1.exe:     contains public_key? False  ciphertext? False
a.png:                contains public_key? False  ciphertext? False
```

None of the other provided files contain the keys used here. There are no cross-references between assets.

### Identifying the cryptographic standards

We matched each field size against known post-quantum cryptographic standards from NIST:

| Field      | Size  | Standard               | Purpose                      |
|------------|-------|------------------------|------------------------------|
| public_key | 2592B | ML-DSA-87 (FIPS 204)   | Signature verification key   |
| signature  | 4627B | ML-DSA-87 + 32 extra   | Signs the ciphertext payload |
| kem_cipher | 1568B | ML-KEM-1024 (FIPS 203) | Encapsulated AES-256 key     |
| ciphertext | 229B  | AES-256-GCM            | Encrypted content            |
| nonce      | 12B   | AES-256-GCM            | Initialization vector        |
| auth_tag   | 16B   | AES-256-GCM            | Integrity check              |

ML-DSA-87 is also known as Dilithium5. ML-KEM-1024 is also known as Kyber-1024. Both were standardized by NIST in 2024 as part of the post-quantum cryptography project. These are the same algorithms being adopted by governments and militaries specifically because they cannot be broken by quantum computers.

The decryption flow requires:

```
Step 1: shared_secret = ML-KEM-1024.Decaps(kem_cipher, private_key)
Step 2: aes_key       = shared_secret
Step 3: plaintext     = AES-256-GCM.Decrypt(aes_key, nonce, ciphertext, auth_tag)
```

The private key for step 1 is 3168 bytes. It was not in any file provided.

### The 32 extra bytes at the end of the signature

ML-DSA-87 signatures are exactly 4595 bytes. The signature field in this file is 4627 bytes -- 32 bytes longer than it should be. We extracted them:

```
$ python3 analyze_passtxt.py [signature overflow]
```

```
sig total: 4627 bytes
ML-DSA-87 standard size: 4595 bytes
Extra: 32 bytes appended:
  0000: e9 f6 26 49 50 84 fc 00 00 00 00 00 00 00 00
  0010: 00 00 00 00 00 00 00 00 02 0d 17 19 24 2b 35 3a
```

The pattern is: 7 non-zero bytes, then 17 zero bytes, then 8 non-zero bytes.

This is a hallmark of a buffer that was allocated but not fully filled before being written to the file. It is a coding bug in whatever tool generated pass.txt.

## All decryption attempts

We tried every angle we could find without the private key:

```
Attempt 1: Extra 32 bytes used as AES-256-GCM key
Result    : Auth failed - MAC check failed
```

```
Attempt 2: All-zero 32-byte key
Result    : Auth failed
```

```
Attempt 3: First 32 bytes of kem_cipher as key
Result    : Auth failed
```

```
Attempt 4: SHA-256 of public_key as key
Result    : Auth failed
```

```
Attempt 5: First 32 bytes of signature as key
Result    : Auth failed
```

```
Attempt 6: Last 32 bytes of signature as key
Result    : Auth failed
```

```
Attempt 7: 12 common password-derived keys (dortress, neelanjan, etc.)
Result    : All failed
```

```
Attempt 8: Tried to regenerate the ML-KEM-1024 keypair using kyber-py
           v1.2.0 with different 48-byte DRBG seeds to find one that
           produces the known public key
```

```
Seed: 48 zero bytes
Generated pk[:8] = ee35c1dcea23b32b
Expected pk[:8] = 3821499044cdf961 -- NO MATCH
```

```
Seed: bytes 0x00 to 0x2F
Generated pk[:8] = b9f73ba7533d90d9
Expected pk[:8] = 3821499044cdf961 -- NO MATCH
```

This last attempt would theoretically work only if we found the exact seed used when the keypair was originally generated. Given Kyber-1024's 256-bit security level this is not computationally feasible.

**SECURITY ASSESSMENT: pass.txt / Post-Quantum Hybrid**

**STATUS: Blocked. ML-KEM-1024 private key not provided. Cannot decrypt.**

**What is good:**

The choice to use ML-KEM-1024 for key encapsulation is excellent. This is current state-of-the-art cryptography. No classical computer and no quantum computer can break Kyber-1024 with the private key unknown. The combination of post-quantum KEM with AES-256-GCM for the actual data encryption is the correct hybrid construction. The ML-DSA-87 signature provides authenticity: the recipient can verify the ciphertext was not tampered with.

**What needs improvement:**

The 32 extra bytes after the signature are a bug. A security-critical file should not have junk bytes appended to a cryptographic signature. This indicates the tool that generated pass.txt has a buffer handling error. While it does not break the security of the scheme, it is unprofessional and in some implementations could be exploited as a side channel or cause verification failures.

There is no public key fingerprint or key ID embedded in the file. If someone has multiple Kyber keypairs they have no way to know which private key to use to open this file.

**RATING: Encryption algorithm choice -- STRONG**

Implementation quality (buffer bug) -- NEEDS IMPROVEMENT

**6. OVERALL SECURITY REPORT CARD**

| Component                         | Rating             |
|-----------------------------------|--------------------|
| -----                             |                    |
| QR encryption (Fernet token)      | STRONG             |
| QR delivery integrity             | WEAK -- no signing |
| a.txt encryption quality          | STRONG             |
| a.txt format (ZEUSBGV obscurity)  | NEEDS IMPROVEMENT  |
| application1.exe key design       | STRONG             |
| application1.exe binary hardening | NEEDS IMPROVEMENT  |
| pass.txt algorithm choice         | STRONG             |
| pass.txt implementation (bug)     | NEEDS IMPROVEMENT  |
| -----                             |                    |
| OVERALL                           | MODERATE-STRONG    |

Summary for Dr. Manna:

The cryptographic algorithms chosen throughout are all appropriate choices. AES-256-GCM, script, Fernet, ML-KEM-1024, and ML-DSA-87 are all good selections. The DLK3 three-factor design in particular is well thought out.

The weaknesses are not in the math. They are in the implementation and distribution decisions: the binary is not stripped of debug information, the ZEUSBGV format relies on secrecy of design rather than strength of key, there is a buffer handling bug in the pass.txt generator, and the QR code can be trivially swapped by anyone with Python.

None of these weaknesses allow the ciphertext itself to be broken. But they are weaknesses that matter in a real-world attack scenario.

7. SCRIPTS AND TOOLS USED

All scripts are in: C:\Users\adversary\Desktop\RevEng\work\

|               |                                                                                                                    |
|---------------|--------------------------------------------------------------------------------------------------------------------|
| file_stats.py | First run. Computed entropy across all three encrypted files and read the first/last 64 bytes.                     |
| recon.py      | Parsed the PNG file chunk by chunk. Also dumped the raw first bytes of a.txt to identify the ZEUSBGV magic header. |
| qr_decode.py  | Decoded the original QR code using OpenCV. Extracted the 96269.in Fernet URL.                                      |
| analyze2.py   | Mapped the a.txt header byte layout. Identified the magic, version, and field offsets.                             |

|                    |                                                                                                                                                                  |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| parse_container.py | Parsed the DLK3 container appended to application1.exe. Extracted the salt, nonce, and container parameters.                                                     |
| pass_recon.py      | Decoded all six JSON fields from pass.txt. Matched field sizes to NIST standards.                                                                                |
| strings_scan.py    | Searched the full binary for printable strings. Found the about section. Confirmed no hidden keys.                                                               |
| ui_strings.py      | Second pass of string extraction focused on finding any credential-like patterns. None found.                                                                    |
| try_decrypt.py     | First decryption attempt on a.txt. AES-256-GCM with SHA-256 derived keys.                                                                                        |
| try_decrypt2.py    | Second attempt. Added ChaCha20-Poly1305, AES-CBC, more offset layouts, more passwords.                                                                           |
| crack_a.py         | Full brute attempt with scrypt and PBKDF2. Multiple salt offsets and all password candidates.                                                                    |
| crack_a2.py        | Added RC4 and no-stored-salt layout variants. Also tested nonce-as-salt configurations.                                                                          |
| crack_a3.py        | Final sweep. PBKDF2 with SHA-256 and SHA-512 at iteration counts from 1000 to 1,000,000 across all byte offset combinations.<br>Output log: crack_a3.out (29 KB) |

#### Reference files generated:

|                     |                                                     |
|---------------------|-----------------------------------------------------|
| main_objdump.txt    | Full objdump -d disassembly (183 KB)                |
| nm.txt              | Full nm symbol table (211 KB)                       |
| crack_a3.out        | Full brute-force attempt log (29 KB)                |
| analyze_passtxt.py  | Full field decode and overflow analysis of pass.txt |
| kyber_seed2.py      | Attempted to regenerate ML-KEM-1024 keypair         |
| nosalt_test.py      | No-salt layout test for a.txt                       |
| chacha_nacl.py      | ChaCha20 and NaCl tests on a.txt                    |
| serial_search.py    | Searched binary for USB serial number patterns      |
| targeted_decrypt.py | 41 passwords x all ciphers in a single run          |

#### Deliverables:

|                                      |                                      |
|--------------------------------------|--------------------------------------|
| a.png                                | QR replaced with jacked message      |
| application1_source_reconstructed.go | Full Go source (17 KB, 23 functions) |
| audit.md                             | This report                          |
| audit.pdf                            | PDF version of this report           |

---

---

Audited by : Vamsi Krishna Orsu  
Email : vamsiorsu.work@gmail.com  
Date : 03 October 2026